Understanding Pattern-Based Visualization of Large-Scale Time Series

Presented by:
Gajara Anjali Rameshbhai (25MCMC03)
Akshay Guru (25MCMC30)

Guided by:
Prof. K.Swarupa Rani

Standard Definition

"A time series is a sequence of observations $x_1, x_2, \ldots, x_n$, each recorded at a specific time (or time interval)." (Box & Jenkins, *Time Series Analysis: Forecasting and Control*, 1976)

What it is:

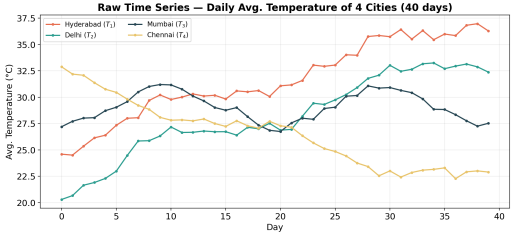
- Data is arranged in time order; nearby observations are often related to each other.
- Data can be collected at equal or unequal time intervals.

Where it is used:

- Finance, healthcare, urban sensing, IoT, mobility & telecom data.

Why we need it :

- Real-world systems continuously produce large amounts of time-based data.
- We need to **visualize, compare and analyze** many series together to find useful trends and patterns

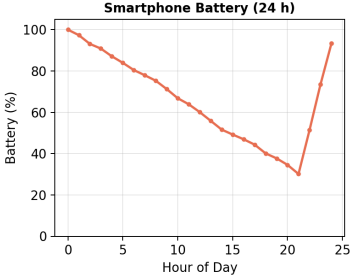


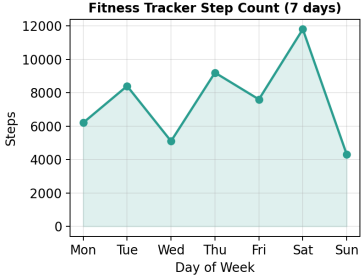
Daily average temperature of four cities

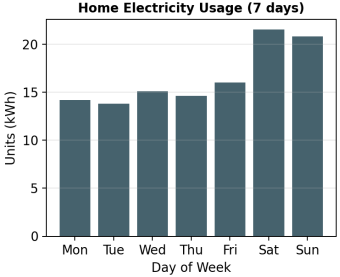
Real-World Examples of Time Series

Here are some simple, everyday examples of time series:

- **Smartphone battery level** – changes during the day as the phone is used and charged; useful for predicting when charging will be needed.
- **Fitness tracker step count** – records daily activity over time; useful for finding active and less-active days.
- **Home electricity usage** – records how much electricity is used each day; useful for finding weekday/weekend patterns and saving energy.



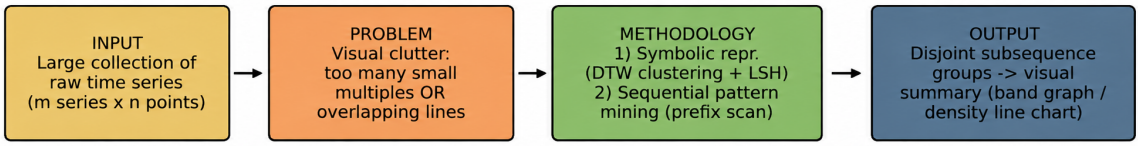




All three are everyday time series. When we plot them over time, we can easily see changes, repeated patterns, and trends.

Problem Statement

Large-scale time-series datasets contain numerous temporal patterns that are difficult to identify and compare manually. As the volume of time-series data increases, conventional visualization becomes insufficient for effectively exploring similarities and discovering meaningful patterns. The key challenge is to automatically identify similar time-series subsequences and organize them into meaningful groups while preserving the important visual patterns in the data.



Input → Problem → Methodologm→ Output (workflow summarized from Chan et al., 2026)

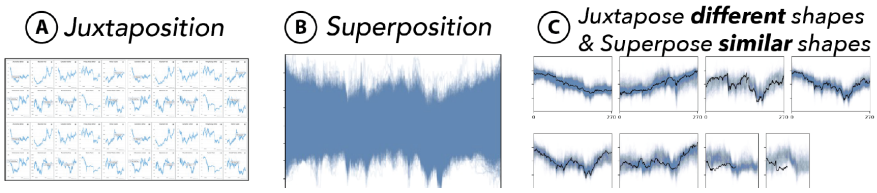
Motivation

Traditionally, multiple time series are shown in two main ways, and both have problems when the number of series becomes large:

- **Juxtaposition** (Scalability issue): each series gets its own chart ⇒ easy to understand, but difficult to use when there are thousands of series because there are too many small charts.
- **Superposition** (Visual Clutter): puts all series in one chart, but many lines overlap and the chart becomes crowded and difficult to read.

TiVy's Idea

Selectively **superpose** time series with *similar visual shapes*, while **juxtaposing** groups with *different* shapes – combining the advantages of both methods.



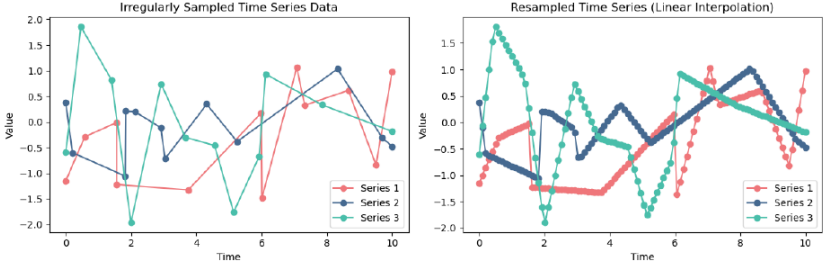
Juxtaposition and superposition both have scalability issues; TiVy juxtaposes different shapes and superposes similar ones (Chan et al., 2026)

Limitation: Irregular Sampling

Limitation. DTW-based clustering, which is an important similarity measure used by TiVy, requires *regularly sampled* data – observations must be recorded at equal time intervals. However, real-world sensor or event data is often *irregularly* sampled, with missing or unevenly spaced timestamps. This creates a problem for the pipeline.

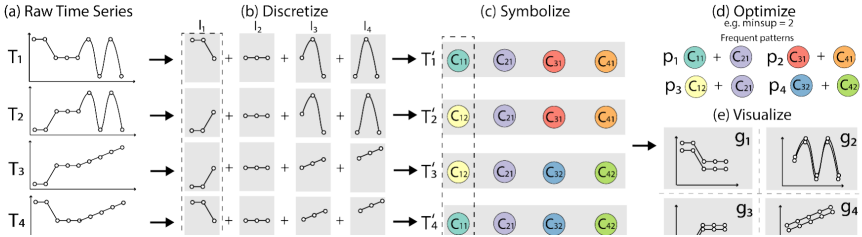
Mitigation

A simple way to handle this is to create a *uniform time grid* and **linearly interpolate** the missing values at each grid point. This puts all series on the same, evenly spaced timeline before symbolic transformation and DTW clustering.



Irregularly sampled raw data (left) resampled onto a uniform grid via linear interpolation (right) (Chan et al., 2026)

The TiVy Pipeline



Pipeline for extracting time series patterns (g_1, g_2, g_3, g_4) *(Chan et al., 2026)*

(a) **Raw** time series $T_1 \dots T_4$

(b) **Discretize**: split each series into equal-length segments $I_1, \dots, I_4$

(c) **Symbolize**: group similar segments in each interval → give them a symbol/color

(d) **Optimize**: find frequent patterns shared by the series (e.g. minsup= 2)

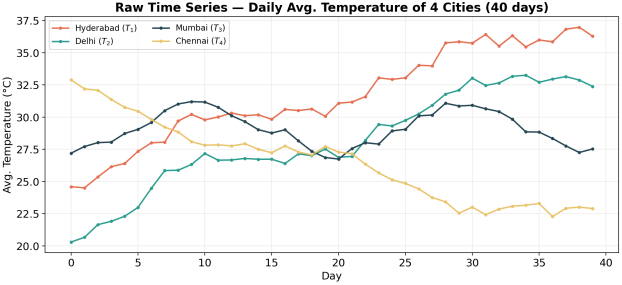
(e) **Visualize**: show the final groups $g_1, \dots, g_4$ as compact charts

Our Example: Raw Time Series

To understand the complete algorithm, we apply it to our own small and easy-to-follow dataset (different from the paper's datasets):

Example Dataset

Daily average temperature (°C) of **4 Indian cities** – Hyderabad (T_1), Delhi (T_2), Mumbai (T_3), Chennai (T_4) – over **40 consecutive days**. Using 40 points and 4 series makes each step of the pipeline easy to understand.



Observations:

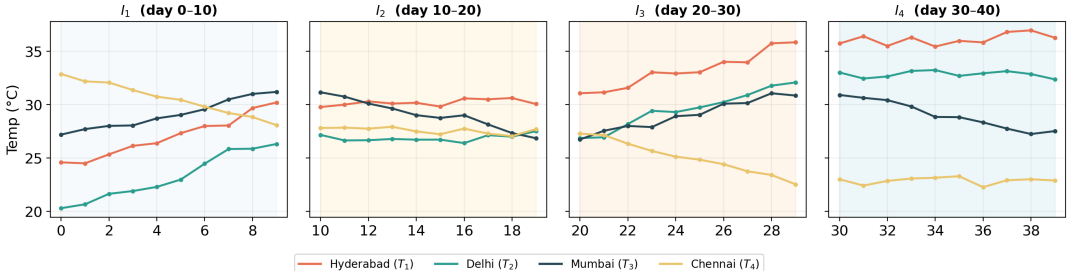
- T_1 (Hyderabad): rises, becomes flat, then rises and becomes flat again
- T_2 (Delhi): has a similar rise/flat shape to T_1, but at a lower temperature
- T_3 (Mumbai): repeatedly goes up and down
- T_4 (Chennai): shows the opposite trend – falls, becomes flat, then falls again

This is the type of dataset TiVy is designed for: some series have similar shapes, while others are different.

Step 1: Discretize

Each series is split into $n' = n/l$ continuous, equal-length segments (Eq. 2 in the paper), $T = s_1 \oplus s_2 \oplus \cdots \oplus s_{n'}$. Here we use a window size of $l = 10$ days, giving 4 sub-intervals I_1, I_2, I_3, I_4 across the 40 days.

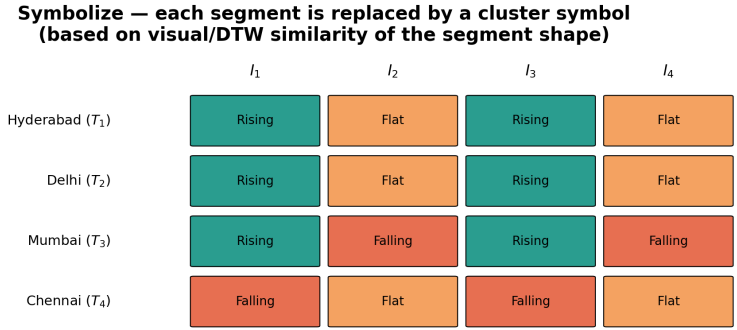
Discretize — split each series into 4 equal sub-intervals (window size l = 10 days)



Each of the 4 city-series split into 4 segments of 10 days each

Step 2: Symbolize

Within each sub-interval, segments from all series are compared for *visual similarity* using DTW distance and grouped into clusters. Each cluster gets a symbol c_{ij}. For easier explanation, we describe each 10-day segment using its main shape –

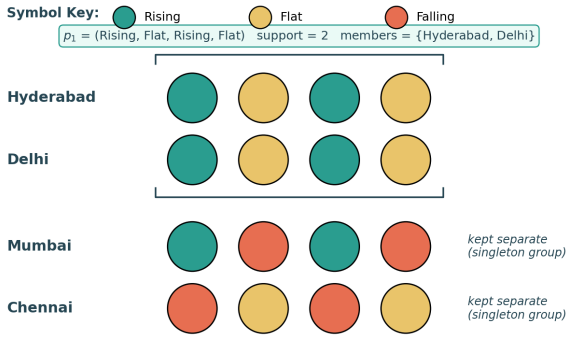


Rising, **Flat**, or **Falling**.

$$T'_{Hyderabad}=(Rising,Flat,Rising,Flat),\ T'_{Delhi}=(Rising,Flat,Rising,Flat),\ T'_{Mumbai}=(Rising,Falling,Rising,Falling),\ T'_{Chennai}=(Falling,Flat,Falling,Flat)$$

Step 3: Optimize

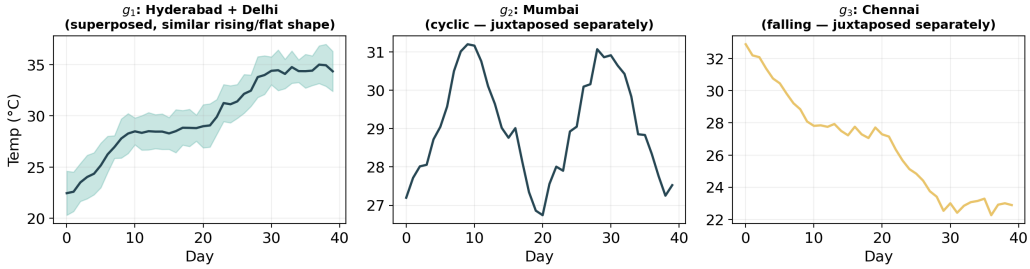
After each series is converted into symbols, TiVy finds **frequent sequential patterns** (Sec. 3.2–3.3) – continuous symbol sequences shared by at least `minsup` series. It then greedily chooses the *longest, non-overlapping* patterns to create the final separate groups $\hat{G} = \{g_1, g_2, \ldots\}$ (Eq. 3–4). Each colored circle below represents one segment's symbol, and the shared pattern must appear in at least `minsup` $= 2$ series.



Hyderabad and Delhi share one frequent 4-symbol pattern p_1; Mumbai and Chennai stay as separate groups

Step 4: Visualize

The separate groups found above are shown as the final **visual summary**: similar series are **superposed** (shown together as a band/central-line chart), while different series are **juxtaposed** in separate small charts.



3 charts instead of 4 – g_1 superposes Hyderabad+Delhi, g_2 and g_3 keep Mumbai and Chennai separate

What This Reveals

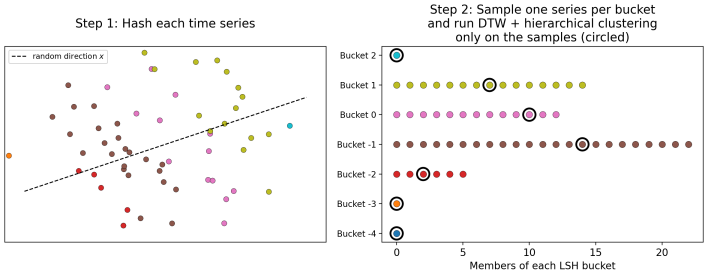
Before this step, all 4 lines were shown together on one chart (Slide 9), so it was difficult to see that Hyderabad and Delhi move in a similar way. After grouping, we can **clearly see**: (1) Hyderabad and Delhi share a similar rising pattern and are shown together, while (2) Mumbai's repeated up-down pattern and Chennai's falling pattern are shown separately and are easier to understand.

Locality-Sensitive Hashing (LSH)

Standard Definition

Locality-Sensitive Hashing is a group of hash functions where similar input points are likely to go to the *same bucket*, while different points are likely to go to different buckets. It is used for fast approximate nearest-neighbour search. (Datar, Immorlica, Indyk & Mirrokni, *Proc. SoCG*, 2004)

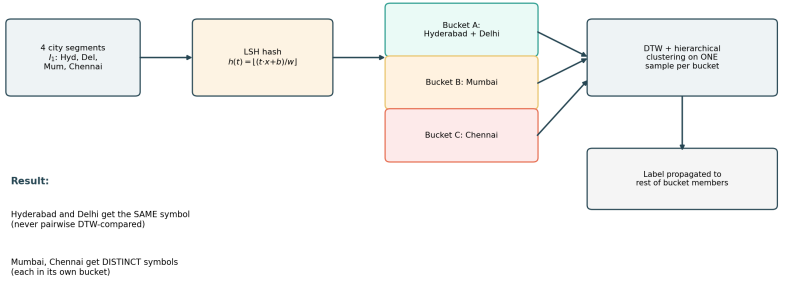
Why it works: series that are close in Euclidean distance (which gives an upper bound for DTW distance) are likely to land in the same bucket. Therefore, hashing provides a cheap first step before the more expensive DTW comparison.



LSH Applied to Our Example

LSH is used to **roughly pre-group** series *before* the expensive DTW step, greatly reducing the $O(n^2)$ number of pairwise DTW comparisons.

1. Each 10-day **segment** of Hyderabad, Delhi, Mumbai, and Chennai (I_1) is hashed. This can be done in one scan without comparing every pair.
2. Hyderabad's and Delhi's rising segments give nearby projected values, so they go into the *same* bucket; Chennai's falling segment goes into a different bucket.
3. From each bucket, **one representative sample** is selected and compared using full **DTW + hierarchical clustering**.
4. The resulting label is **shared** with the other members of the bucket – so Hyderabad and Delhi receive the same symbol without needing a separate DTW comparison between them.



What happens on our example: 4 segments → hashed into 3 buckets → one DTW comparison per bucket → labels propagated to all members

DTW Clustering

Standard Definition

Dynamic Time Warping is an algorithm for measuring similarity between two time sequences that may occur at different speeds or slightly different times. It finds the best non-linear alignment (warping path) between their points and minimizes the total distance. (Berndt & Clifford, *AAAI Workshop*, 1994)

Why DTW (and not Euclidean Distance)? DTW can handle small shifts in time and can align series that have a similar *shape* even when their changes happen at slightly different times. Euclidean Distance compares points at the same positions and does not handle this time shift as well.

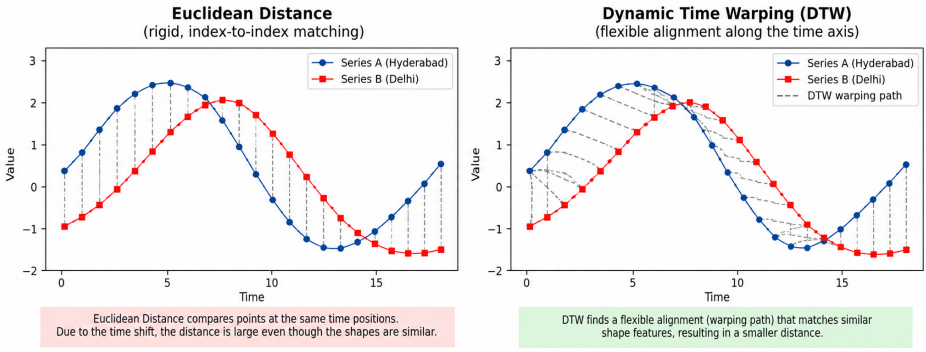
What makes DTW useful for Time Series?

- **Shape-based similarity:** Focuses on the overall pattern of a time series rather than only comparing values at the same time positions.
- **Handles different timing:** Two similar patterns can still be considered similar when one occurs earlier, later, or at a different speed.
- **Works with subsequences:** Useful for finding similar patterns within longer time series.

Role in TiVy: DTW is used to identify similarity between time-series subsequences and supports the clustering of subsequences with similar temporal patterns. This helps TiVy form meaningful groups for large-scale visualization.

DTW Applied to Our Example

Consider the I_1 segments (days 0–10) of Hyderabad (T_1) and Delhi (T_2): both rise, but Delhi's rise starts a little later and is steeper. A fixed point-to-point (Euclidean) comparison may make them look more different than they really are. DTW can shift the time alignment to match important shape features (start, middle rise, plateau), giving a smaller distance that better reflects their similar shape.



Euclidean (rigid) vs. DTW (flexible) alignment – how Hyderabad's and Delhi's rising segments get matched

This is why, in our example (Slides 11–13), Hyderabad's and Delhi's segments received the *same* symbol in every interval and were finally superposed into one group g_1, even though their changes were not perfectly synchronized.

Why DTW Clustering is Suitable

DTW-based hierarchical clustering is chosen over other clustering approaches because time series in a line chart must be grouped by *visual shape*, not by rigid point-to-point distance.

Method	Distance	Shifts?	Misalign?	Suitable?	Main Limitation
Euclidean k-Means	Point-to-point	No	No	Poor	Overstates distance for warped series
Hierarchical (Eucl.)	Point-to-point	No	No	Poor	Same issue; also $O(n^2)$ merges
DBSCAN	Euclidean/density	No	No	Poor	Needs ε, hard to set for series
k-Medoids	Eucl. or DTW	If DTW	If DTW	Fair	Needs pre-specified k
DTW + Hierarchical	**DTW (warped)**	**Yes**	**Yes**	**Good**	Quadratic cost – mitigated via LSH

DTW is suitable here because...

series with the same shape but slightly different timing must still be recognized as similar for a clean line-chart summary. DTW's warping path captures this shape similarity, while gap statistics removes the need to pre-specify k or a distance threshold.

Could one single algorithm replace LSH + DTW?

The paper tests this directly (Fig. 8) using **plain DTW**, **SoftDTW on GPU**, and **FastDTW** without LSH pre-grouping. These methods are faster than basic DTW, but they are still slower than LSH+DTW on CPU and do not fit as naturally with the pattern-mining step. Therefore, one algorithm can give a similar result, but the two-stage LSH+DTW approach is a better fit for this task. This also connects to the paper's future work on *learned vision-based similarity* (Slide 21).

Datasets Used in Evaluation

1. Synthetic Data

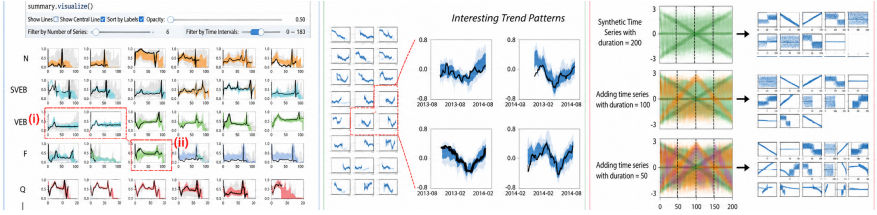
A specially created dataset with **6 known shape classes** (cyclic, normal, increasing, decreasing, upward shift, downward shift) across different time intervals and durations. It is used to check whether TiVy finds the *correct, known* patterns (ground truth is available).

2. Stock Time Series

4,470 daily adjusted stock-price series from **NASDAQ (2015–2016)**, with industry-sector labels. It is used for the portfolio-construction example.

3. ECG Dataset

100,000 patient heartbeat signals, with about 200 values in each signal, grouped into **5 AAMI heart-condition groups**. It is used to test scalability and usefulness for ECG data.



Key Contributions

1. **Visual Similarity-Based Subsequence Clustering**
   TiVy groups time-series **subsequences with similar visual shapes and temporal patterns**, instead of treating the entire time series as one unit.

2. **Efficient Pattern Extraction**
   It converts time series into **symbolic sequences** using visual similarity and uses **sequential pattern mining** to efficiently discover common subsequence patterns.

3. **Scalable Interactive Visualization**
   TiVy provides an **interactive visualization system** that can summarize and explore very large collections of time series with reduced visual complexity.

4. **Evaluation on Real and Synthetic Data**
   The approach is evaluated using **synthetic data, stock-market data, and ECG signals** to demonstrate its effectiveness, scalability, and usefulness in practical scenarios.

Overall Contribution

TiVy combines **visual similarity, DTW-based clustering, symbolic representation, and sequential pattern mining** to produce compact and readable summaries of large-scale time-series data.

Conclusion

By studying the TiVy pipeline and applying it to our own 4-city temperature example, we understood the following main ideas from the research paper:

- Visual similarity can be found **locally, within each time interval**, so two series do not need to be identical everywhere. Hyderabad and Delhi were grouped even though their actual temperatures were different.
- Converting series into **symbolic sequences** changes a costly continuous shape-comparison problem into a faster, discrete **sequential pattern mining** problem. This makes the large visualization problem easier to handle computationally.
- A **two-stage similarity search** (fast LSH pre-grouping $\rightarrow$ DTW only on selected samples) helps the method scale from our small 4-series example to **100,000+ real ECG series** while keeping the visual results useful.
- Most importantly, applying each stage to our own dataset made the algorithm easier to understand. We could see step by step *why* Hyderabad and Delhi were grouped, while Mumbai and Chennai were kept separate.

Overall

TiVy addresses the juxtaposition-vs-superposition problem by using similarity-based sequential-pattern clustering. Our worked example helped us understand how the method creates a clearer visual summary.

Future Work

The authors identify four possible directions for future work:

1. **Extend TiVy to multivariate time series** – to analyze situations with several related variables, such as multiple sensors measuring different things at the same time.
2. **Support interactive refinement / sketching** – allowing experts to guide pattern discovery directly, for example by drawing a query.
3. **Explore modern vision models** instead of the DTW clustering pipeline – learning visual similarity directly from time-series images or projections, which may improve scalability.
4. **Apply the technique to other domains** beyond ECG and stock-market data.

Our Chosen Direction

We plan to focus on **Direction 1 – extending TiVy to multivariate time series**. Many real-world sensor and IoT systems record several related variables at the same time. Studying them together may reveal patterns that a single-variable approach cannot capture.

References

1. G. Y.-Y. Chan, L. G. Nonato, T. Palpanas, C. T. Silva, J. Freire, "TiVy: Time Series Visual Summary for Scalable Visualization," *IEEE Transactions on Visualization and Computer Graphics*, vol. 32, no. 1, pp. 232–242, Jan. 2026. (Primary paper – source of all paper figures reproduced in this presentation)
2. G. E. P. Box, G. M. Jenkins, *Time Series Analysis: Forecasting and Control*, 1976. (Standard definition of time series, Slide 2)
3. D. J. Berndt, J. Clifford, "Using Dynamic Time Warping to Find Patterns in Time Series," *AAAI Workshop*, 1994. (DTW definition, Slides 16–17)
4. M. Datar, N. Immorlica, P. Indyk, V. S. Mirrokni, "Locality-Sensitive Hashing Scheme Based on p-Stable Distributions," *Proc. SoCG*, pp. 253–262, 2004. (LSH definition, Slide 14)
5. R. Tibshirani, G. Walther, T. Hastie, "Estimating the Number of Clusters in a Data Set via the Gap Statistic," *J. R. Stat. Soc.*, 63(2):411–423, 2001. (Gap-statistic clustering, Slide 16)
6. J. Pei et al., "PrefixSpan: Mining Sequential Patterns Efficiently by Prefix-Projected Pattern Growth," *Proc. ICDE*, 2001. (Sequential pattern mining baseline referenced in the paper)
7. J. Lin, E. Keogh, S. Lonardi, B. Chiu, "A Symbolic Representation of Time Series (SAX)," *Proc. ACM SIGMOD Workshop*, 2003. (Symbolic time-series representation, related work)
8. MIT-BIH Arrhythmia Database – `https://www.physionet.org/content/mitdb/1.0.0/` (ECG dataset, Slide 19)

Image sources: Figures reproduced directly from the primary paper (Chan et al., 2026) are used for academic, non-commercial educational purposes (Slides 5, 6, 8, 19). The other figures – the introduction graph, everyday examples, problem-statement flowchart, worked-example graphs, and DTW/LSH illustrations – were created by the presenters for this presentation (Slides 2–4, 9–18).